



UNIVERSITY OF
LIVERPOOL

COMP575

Tutorial 8

Genetic Algorithms

Tutorial slides created by Botond Hamori

Combinatorial Problems

- Finding the optimal solution is often not the optimal strategy
 - Due to costs relating to
 - Computation
 - Time
 - Storage
- We often don't know the true extent of the difficulty of combinatorial problems
 - P, NP, NP-complete, NP-hard
 - Relates to how easy it is to find an optimal solution (if at all possible) and how quickly it can be verified
 - Famous unsolved problem: P versus NP
 - Can a problem be solved in polynomial time and can the solution be verified in polynomial time?
 - One of the Millennium Prize problems (worth \$1,000,000)



Motivation

- Since the extent of the solution space isn't always known to us, we can use an algorithm that goes from an initial guess
- Challenge:
 - How do we encode a problem?
 - What is our objective?
 - How do we measure the quality of our solution?
- Genetic algorithms encode a problem often in binary representation
- We define our objective function
 - The function we want to solve (i.e. minimisation or maximisation)
- We define our fitness function
 - Usually calculated using the objective function to determine if the solution is better or worse than previous combinations



Genetic Algorithm (GA)

- Metaheuristic algorithm
 - Aims to explore as much of the problem space as possible
 - Some complex problems cannot be solved, their optimum can only be approximated
- The process in a nutshell:
 1. Start with a population of random solutions to a problem
 2. Evaluate each individual's "fitness"
 3. Generate new population
 4. Repeat until n generations

Core Concepts – Representation of the Solution

- Population
 - A set of candidate solutions. Each solution is encoded as a *chromosome*
- Chromosome
 - A single candidate solution. It is a string of genes encoding one specific solution
- Gene
 - A single value in a chromosome (e.g. 0)
- Allele
 - A specific value a gene can take (for example, in binary encoding, the alleles are 0 and 1)

Core Concepts – Genetic Operators

- Crossover
 - Combine two parent chromosomes to create one or more (usually 2) offspring. 3 types of crossover:
 - Single-point: genes are swapped at a specific point
 - Two-point: genes are swapped between two points
 - Uniform: each of the genes is randomly inherited from the parent
- Mutation
 - Randomly flip the value of a gene in a chromosome
- Inversion
 - Reverse the order of a sequence of genes



Fitness function

- Evaluates how well a candidate solution solves the problem
- Selection: chooses chromosomes for reproduction based on their fitness function
- Selection strategies:
 - Tournament: pick a subset of solutions and select the best among them
 - Roulette wheel: select a candidate solution with a probability directly proportional to their fitness
 - Rank: sort the population by fitness and assign selection probabilities based on the rank. The selection probability is computed using a mapping function
 - The mapping function controls the selection pressure. There is “higher pressure” of selection on top candidates
 - Elitism: sort the population by fitness and select the top n candidates

The Travelling Salesman Problem

- Aim: minimise the total distance to visit each vertex in a graph
 - The result must be a Hamiltonian (acyclic) graph (each vertex to be visited exactly once)
- All vertices are connected (unless constraints are imposed)
- The problem is NP-hard
 - Exponential growth in complexity as we increase the number of vertices in our graph
 - Potential number of path combinations: $\frac{(n-1)!}{2}$
 - 5 vertices: 12 combinations
 - 10 vertices: 181,440 combinations
 - 100 vertices: 4.666×10^{155} combinations
- Becomes unfeasible to brute force an optimal solution
 - In many cases, it would take longer than the age of the universe



Worked Example – Maximise $f(x) = x^2$

- Apply GA maximisation for the function $f(x) = x^2$ where $x \in [0,31]$

Initially (generation zero):

String no.	Initial population	x Value	Fitness $f(x) = x^2$	$Prob_i$	Expected count	Actual count
1	0 1 1 0 1	13	169	0.14	0.58	1
2	1 1 0 0 0	24	576	0.49	1.97	2
3	0 1 0 0 0	8	64	0.06	0.22	0
4	1 0 0 1 1	19	361	0.31	1.23	1
Sum			1170	1.00	4.00	4
Average			293	0.25	1.00	1
Max			576	0.49	1.97	2

Worked Example – Maximise $f(x) = x^2$

- Representation: 5-digit binary string representing the x value ($2^5 = 32$, so in the range 0-31)
- The fitness function is defined as $f(x) = x^2$
- Population of 4 individuals
- Notes on columns
 - String no.: the label used to identify a chromosome
 - Initial population: the population at the start of the current generation before performing any genetic operators
 - x value: the value being encoded by the chromosome
 - $Prob_i$: selection probability
 - (fitness of individual / total fitness of population)
 - Expected count: number of times the individuals is expected to be selected
 - Actual count: actual number of selections

Initially (generation zero):

String no.	Initial population	x Value	Fitness $f(x) = x^2$	$Prob_i$	Expected count	Actual count
1	0 1 1 0 1	13	169	0.14	0.58	1
2	1 1 0 0 0	24	576	0.49	1.97	2
3	0 1 0 0 0	8	64	0.06	0.22	0
4	1 0 0 1 1	19	361	0.31	1.23	1
Sum			1170	1.00	4.00	4
Average			293	0.25	1.00	1
Max			576	0.49	1.97	2

Worked Example – Maximise $f(x) = x^2$

- Selection
 - Chromosome 2 has the highest fitness
 - Select it and the first candidate with an actual count > 0
 - No elitism (yet)
 - Then select chromosome 2 again and find the next candidate with an actual count > 0

Initially (generation zero):

String no.	Initial population	x Value	Fitness $f(x) = x^2$	$Prob_i$	Expected count	Actual count
1	0 1 1 0 1	13	169	0.14	0.58	1
2	1 1 0 0 0	24	576	0.49	1.97	2
3	0 1 0 0 0	8	64	0.06	0.22	0
4	1 0 0 1 1	19	361	0.31	1.23	1
Sum			1170	1.00	4.00	4
Average			293	0.25	1.00	1
Max			576	0.49	1.97	2

Worked Example – Maximise $f(x) = x^2$

- Crossover
 - Single-point crossover
 - Randomly identify a crossover point for each chromosome pair
 - These points remain the same between the pairs
 - The first offspring receives the first parent's first sequence of genes, then inherits the rest from the second parent
 - The second offspring receives the second parent's first sequence and receives the rest from the first parent.
 - For example, chromosomes 1 and 2 create these two offspring:

0	1	1	0	0
1	1	0	0	1

Crossover (assume $p_c=1$):

String no.	Mating pool	Crossover point	Offspring after xover	x Value	Fitness $f(x) = x^2$
1	0 1 1 0 1	4	0 1 1 0 0	12	144
2	1 1 0 0 0	4	1 1 0 0 1	25	625
2	1 1 0 0 0	2	1 1 0 1 1	27	729
4	1 0 0 1 1	2	1 0 0 0 0	16	256
Sum					1754
Average					439
Max					729

Worked Example – Maximise $f(x) = x^2$

- Mutation
 - There is a probability that one of the genes in a chromosome randomly mutates
 - This is useful for avoiding getting stuck at a local optimum

Mutation:

String no.	Offspring after xover	Offspring after mutation	x Value	Fitness $f(x) = x^2$
1	0 1 1 0 0	1 1 1 0 0	28	784
2	1 1 0 0 1	1 1 0 0 1	25	625
2	1 1 0 1 1	1 1 0 1 1	27	729
4	1 0 0 0 0	1 0 1 0 0	20	400
Sum				2538
Average				634.5
Max				784

Diagnostics Metrics

- We saw 3 metrics in this example: *sum*, *average*, and *maximum*
- Sum
 - Influences selection probability
 - Higher sum means stronger candidates in population
 - If too low, mutation can be increased either before or during the algorithm
 - This metric should increase over time (for maximisation problems)
- Average
 - Reflects how well the selection and crossover operators perform
 - If it stays low or plateaus early, we lose diversity in candidate solutions
 - Leads to premature convergence (the algorithm gets stuck at a local optimum)
- Maximum
 - The best solution in the population. It helps with tracking convergence progress
 - If there is no improvement for several iterations, the algorithm can be stopped early
 - If stagnates early, increasing mutation probability or changing crossover strategy can help the algorithm produce better solutions

Example Applications

- Neural Networks
 - Architecture optimisation and feature selection
 - Objective: minimise the loss function and/or find an optimal balance between model complexity and convergence speed
- Game Theory
 - Encode strategies as genes
 - Objective: payoff maximisation/finding Nash equilibria/etc
- RNA Structure Prediction
 - Encode RNA sequences (chromosomes represent a possible secondary structure)
 - Objective: minimise energy
- Robotics
 - Path planning and trajectory optimisation
 - Objective: minimise the total distance between a set of points
 - Evolutionary robotics is conceptually similar to GA

Other Optimisation Algorithms

- Differential evolution
- Simulated Annealing
- Ant colony optimisation
- Particle swarm optimisation

Q&A